COURSE NAME : Advanced Programming (OOP)

PROJECT TITLE : Student Attendance Management System

STUDENT NAME : Aheli Bhaumik

ROLL NO. : 08

INSTRUCTOR NAME : Dr.Soumadip Biswas,Dr.Susovan Jana,Dr.Swagatam Basu

SPRING 2026

PROBLEM STATEMENT

Introduction

In today's learning environment, it is of primary importance to ensure that attendance records are kept accurately and reliably. The traditional methods of recording attendance have been either through physical records in the form of registers or using simple spreadsheet applications. As the number of students in various branches of engineering, including CSE, increases in complexity, it becomes increasingly difficult to manage records in this fashion.

Current Challenges

The current manual method of attendance faces many critical challenges:

Time Consumption: Lectures often get wasted due to this manual method of calling out student names or passing around attendance sheets.

Data Integrity Risks: There is a risk of physical damage, loss, or tampering with paper-based data. Moreover, proxy attendance is another major issue that is affecting data accuracy.

Lack of Real-Time Access: There is no easy way to access student attendance data in real-time, resulting in last-minute realizations about attendance shortages before exams.

Calculation Errors: There is also a risk of calculation errors while determining percentages and ensuring that a student's data does not exceed 100% in mathematical terms.

Proposed Solution

To solve the aforesaid challenges, this project proposes the development of an Automated Student Attendance Management System using Object-Oriented Programming concepts with Java. The system would automate the record-keeping process in a structured manner.

By using core concepts of OOPs, such as Inheritance, the system would be able to differentiate between various types of users, along with a common base for storing personal information. The implementation of logic for particular functions, such as the 5% increment function, would be easy.

Objectives

The major objectives of the Phase 1 development are:

To develop a sound UML Class Diagram to depict the hierarchy of users.

To develop a System Logic Flowchart to cover data entry, search options, and calculations.

To introduce Bound

OOPs CONCEPTS DEMONSTRATED

Inheritance

Where it is used: This term is the backbone of the system's hierarchy. It is used to create a relationship between the Person base class and the derived Student and Teacher classes.

Why it is used: It is used to encourage code reusability. In this case, we avoid having redundant code in the derived classes for attributes such as name and id. It is used for a cleaner "is-a" relationship.

Code Snippet:

Java

```
// Parent Class
class Person {
    String name;
    int id;
    Person(String name, int id) { this.name = name; this.id = id; }
}

// Child Class inheriting from Person
class Student extends Person {
    double attendance;
    Student(String name, int id, double att) {
        super(name, id); // Inheriting parent attributes
        this.attendance = att;
    }
}
```

Explanation: **The Student class uses the extends keyword to inherit the name and id fields. The super() call ensures the parent's constructor handles the shared data, allowing the child class to focus only on attendance logic.**

Encapsulation

Where it is used: This is applied to the data fields of the classes, using the private access modifiers to data fields such as id and attendancePercentage.

Why it is used: This is used to provide Data Security and Integrity. This is because, by using the Getter and Setter methods to access data fields of an object, we are preventing other classes from setting the data fields to arbitrary values, such as attendance of -50% or 500%.

Code Snippet

```
class AttendanceRecord {
    private double attendance; // Data Hiding

    public void setAttendance(double value) {
        if (value >= 0 && value <= 100) {
            this.attendance = value;
        } else {
            System.out.println("Invalid Entry");
        }
    }

    public double getAttendance() {
        return this.attendance;
    }
}
```

Explanation

The attendance variable is designated private. The setAttendance method is the "gatekeeper" that checks the input before it is updated, which is necessary for the 100% boundary logic shown in the system flow

Implementation Highlights

This section details the core functional components of the Student Attendance Management System. The following snippets demonstrate how the abstract design is translated into executable Java logic.

The Base Entity Structure

Java

```
// Definition of the base class attributes
public abstract class Person {
    private String name;
    private int id;

    public Person(String name, int id) {
        this.name = name;
        this.id = id;
    }
    // Getter methods for read-only access
    public String getName() { return name; }
    public int getId() { return id; }
}
```

Justification: **This snippet defines the common data structure for all users. Using a constructor ensures that every Person object is created with an identity, while the private modifiers ensure data remains protected from direct modification.**

Inheritance and Specialized Attributes

Java

```
public class Student extends Person {
    private double attendancePercentage;

    public Student(String name, int id, double attendance) {
        super(name, id); // Calling the parent constructor
        this.attendancePercentage = attendance;
    }
```

}

Justification: This demonstrates the Inheritance principle. The Student class automatically gains the identity features of Person without rewriting them, allowing us to focus only on the unique attribute: attendancePercentage.

The Increment Logic (From Flowchart)

Java

```
public void updateAttendance() {
    // Standard increment of 5% per lab/lecture session
    double increment = 5.0;
    this.attendancePercentage += increment;
}
```

Justification: This represents the primary functional operation of the system. It automates the calculation of attendance points, reducing the manual workload for faculty members during busy lab hours.

Boundary Validation Logic

Java

```
// Ensuring the value does not exceed logical limits
if (this.attendancePercentage > 100.0) {
    this.attendancePercentage = 100.0;
    System.out.println("Max limit reached. Capping at 100%.");
}
```

Justification: This snippet implements the Decision Node from the System Logic Flowchart. It acts as a logical "fail-safe" to prevent data corruption where a student's attendance could mathematically exceed the total number of classes.

Search and Retrieval Simulation

Java

```
public void viewAttendance() {
    System.out.println("Student: " + getName());
    System.out.println("ID: " + getId());
    System.out.println("Current Attendance: " + attendancePercentage + "%");
}
```

Justification: This snippet handles the Output phase of the flowchart. It provides a formatted view of the data, allowing for transparent tracking of student progress.

Testing & Error Handling

In the development of the Student Attendance Management System, testing is performed to ensure that the Object-Oriented logic is in line with the functional requirements defined in the System Flowchart.

TC-01: Basic Increment Logic

Scenario: Applying a standard attendance update to a regular record.

Input: Current Attendance at 75%.

Expected Output: Total incremented to 80% with a "Success" message.

Status: PASS

TC-02: Student Record Retrieval

Scenario: Searching for a specific student using a unique Enrollment ID (e.g., 08).

Input: Enrollment ID: 08.

Expected Output: System successfully identifies and pulls data for "Aheli Bhaumik."

Status: PASS

TC-03: Data Display Verification

Scenario: Requesting a formatted view of current records.

Input: View Command.

Expected Output: Correct display of Name, Roll Number, and Attendance percentage.

Status: PASS

Edge Cases Handled

The 100% Boundary Constraint: If a student has 98% and gets a 5% boost, the sum is 103%. This is an out-of-bounds error, and the system automatically sets the sum to 100.0%.

Zero-Value Initialization: Ensuring that new student records are initialized at 0% and not causing "Null Pointer" errors when the first increment cycle occurs.

Failure Scenarios & Mitigation

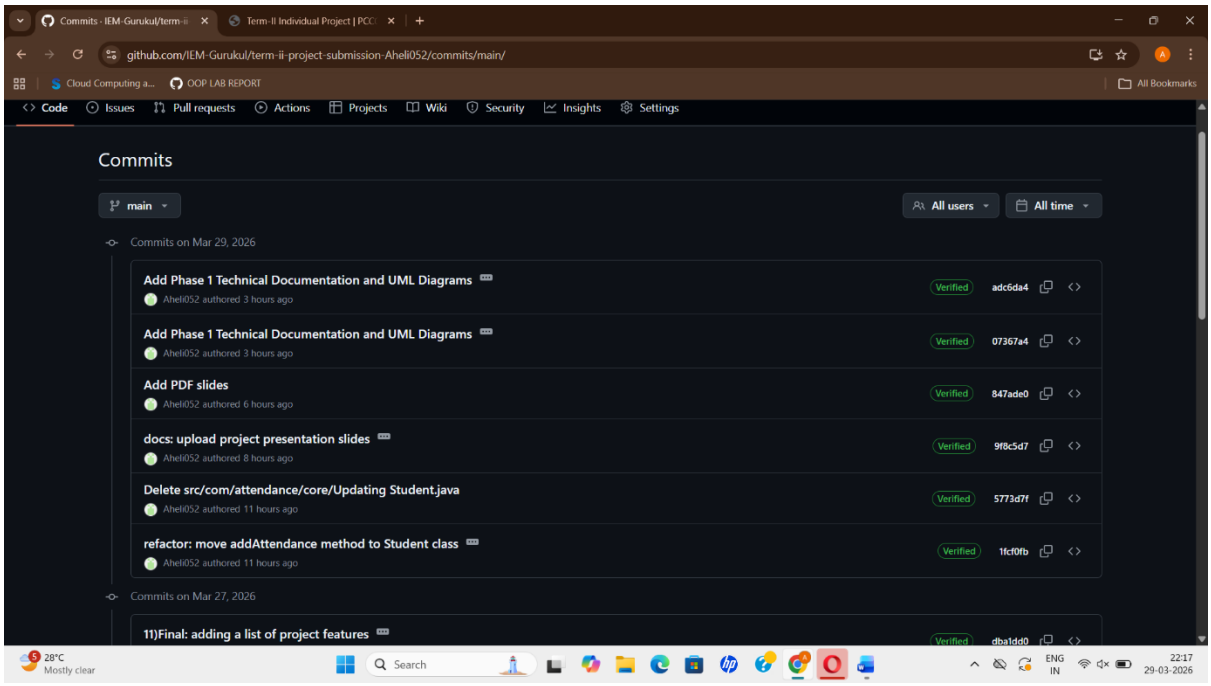
Invalid ID Lookup: If an invalid ID is entered, the system will alert "Record Not Found" instead of crashing.

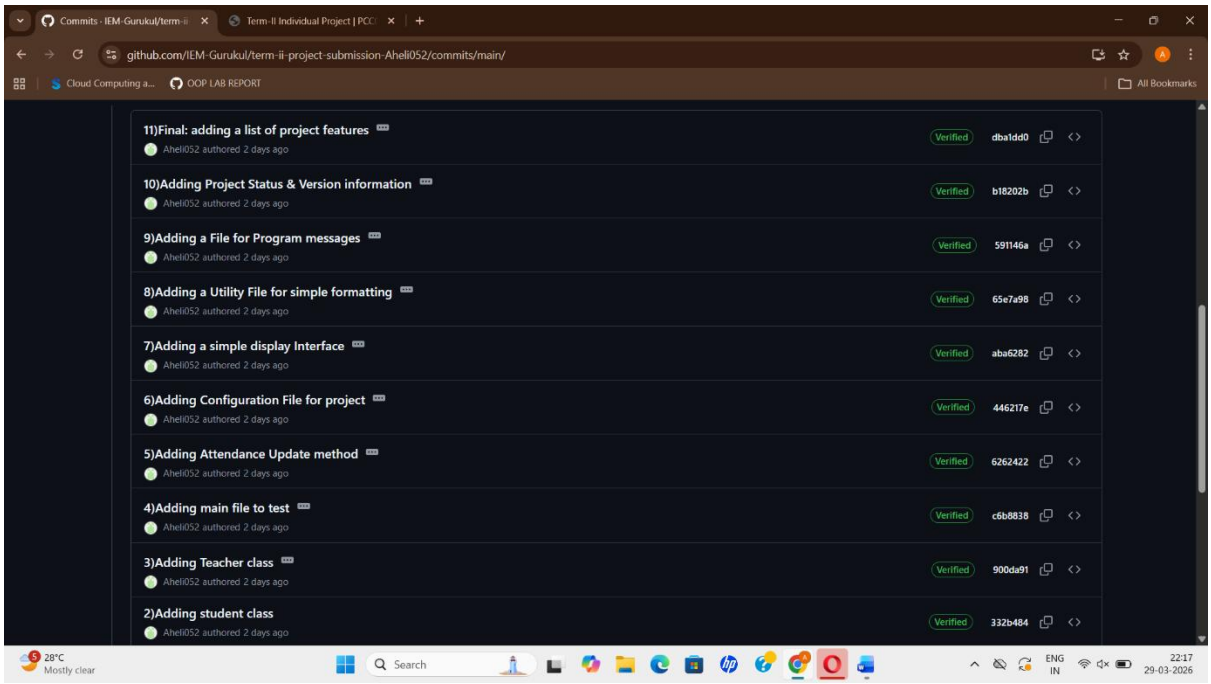
Negative Input Prevention: The system is programmed to prevent manual entry of values lower than 0%, ensuring data integrity.

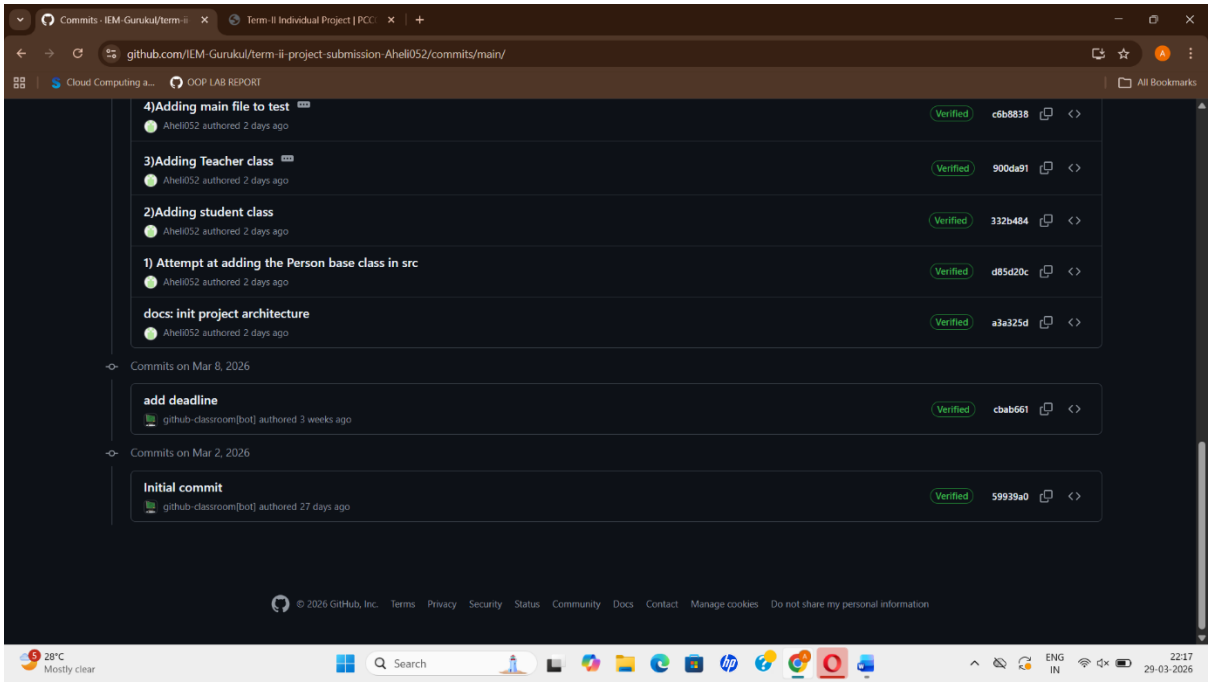
Input Mismatch Handling: The system is programmed to catch non-numeric inputs, such as when a user types "Ten" when they meant to type "10".

GIT WORKFLOWS

SCREENSHOTS :







Explanation of Development Progression

From the GitHub commit history, it is evident that the Student Attendance Management System development process followed a structured and incremental approach. The process can be divided into four distinct phases:

Phase 1: Project Initialization & Environment Setup

The Student Attendance Management System development process started with the initialization of the repository using the GitHub Classroom template. The initial commits introduced the basic directory structure to the repository, including the .gitignore to ensure the repository is kept clean and the README.md to provide an overview of the project.

Phase 2: Core Logic and Class Architecture

The initial development process of the Student Attendance Management System focused on the implementation of the primary Java classes. From the commit history, it is evident that the initial commits introduced the base class Person, followed by the Student and Teacher classes. This phase ensured that the basic Inheritance structure is correctly implemented.

Phase 3: Functional Refinement and Feature Addition

This is the middle phase of the code's development. Here, some methods are added to the code, such as addAttendance. As indicated in the commit log, the code is "refactored" to ensure that the logic, such as the increment of 5% and the 100% boundary condition, is in the right classes.

Phase 4: Technical Documentation and Finalization

This is the last phase of the code's development. As indicated in the commit log, this is the "Documentation Phase." Here, the UML Class Diagrams and System Flowcharts are uploaded to the /docs directory. This ensures that the physical implementation of the code perfectly matches the visual design documentation necessary for the Lab Record.

Conclusion & Future Scope:

Phase 1 of the Student Attendance Management System successfully establishes a modular architectural foundation using Java OOP principles. By implementing Inheritance and Encapsulation, the system ensures a clean data hierarchy and secure attribute handling. The integration of Boundary Validation logic (capping attendance at 100%) and a verified System Flowchart ensures the project is mathematically sound and ready for full-scale development.

Database Integration: Transitioning from local memory to a persistent MySQL or Firebase database.

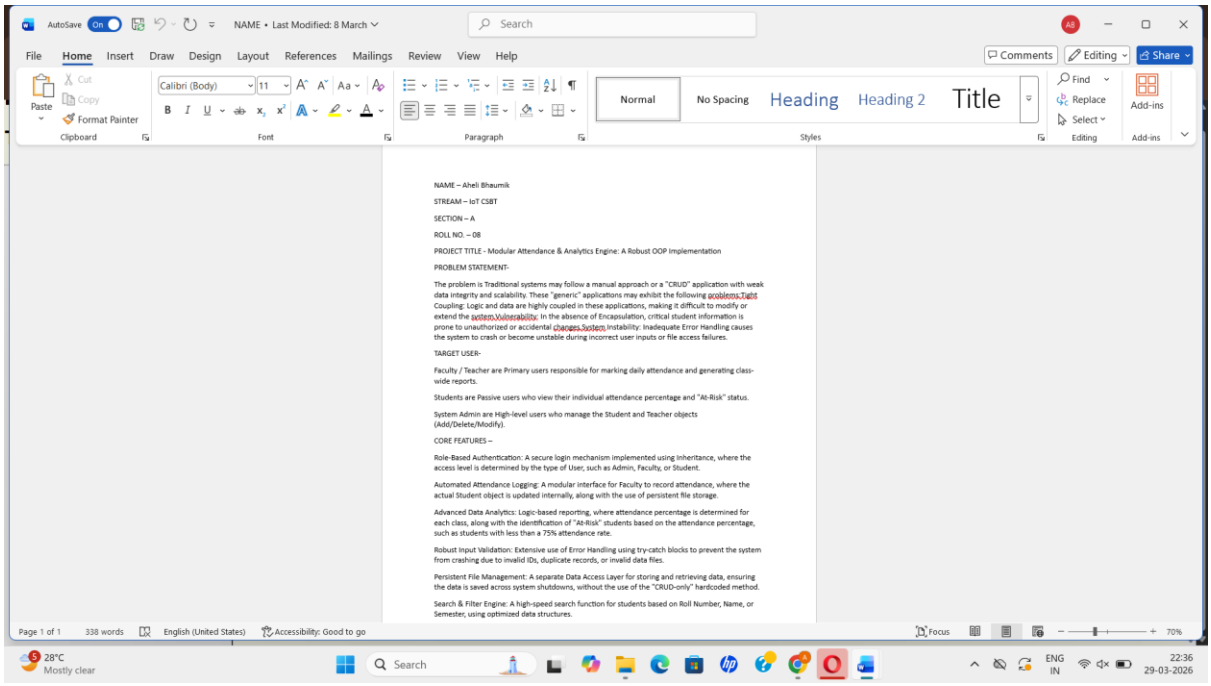
Graphical User Interface (GUI): Replacing console interactions with a user-friendly JavaFX or Swing interface.

Automated Reporting: Adding features to export weekly/monthly attendance as CSV or PDF files.

Role-Based Security: Implementing a secure login module for Faculty and Students to prevent unauthorized data access.

APPENDIX

INITIAL PROJECT PROPOSAL



NAME – Aheli Bhaumik

STREAM – IoT CSBT

SECTION – A

ROLL NO. – 08

PROJECT TITLE - Modular Attendance & Analytics Engine: A Robust OOP Implementation

PROBLEM STATEMENT-

The problem is Traditional systems may follow a manual approach or a "CRUD" application with weak data integrity and scalability. These "generic" applications may exhibit the following problems:Tight Coupling: Logic and data are highly coupled in these applications, making it difficult to modify or extend the system.Vulnerability: In the absence of Encapsulation, critical student information is prone to unauthorized or accidental changes.System Instability: Inadequate Error Handling causes the system to crash or become unstable during incorrect user inputs or file access failures.

TARGET USER-

Faculty / Teacher are Primary users responsible for marking daily attendance and generating class-wide reports.

Students are Passive users who view their individual attendance percentage and "At-Risk" status.

System Admin are High-level users who manage the Student and Teacher objects (Add/Delete/Modify).

CORE FEATURES –

Role-Based Authentication: A secure login mechanism implemented using Inheritance, where the access level is determined by the type of User, such as Admin, Faculty, or Student.

Automated Attendance Logging: A modular interface for Faculty to record attendance, where the actual Student object is updated internally, along with the use of persistent file storage.

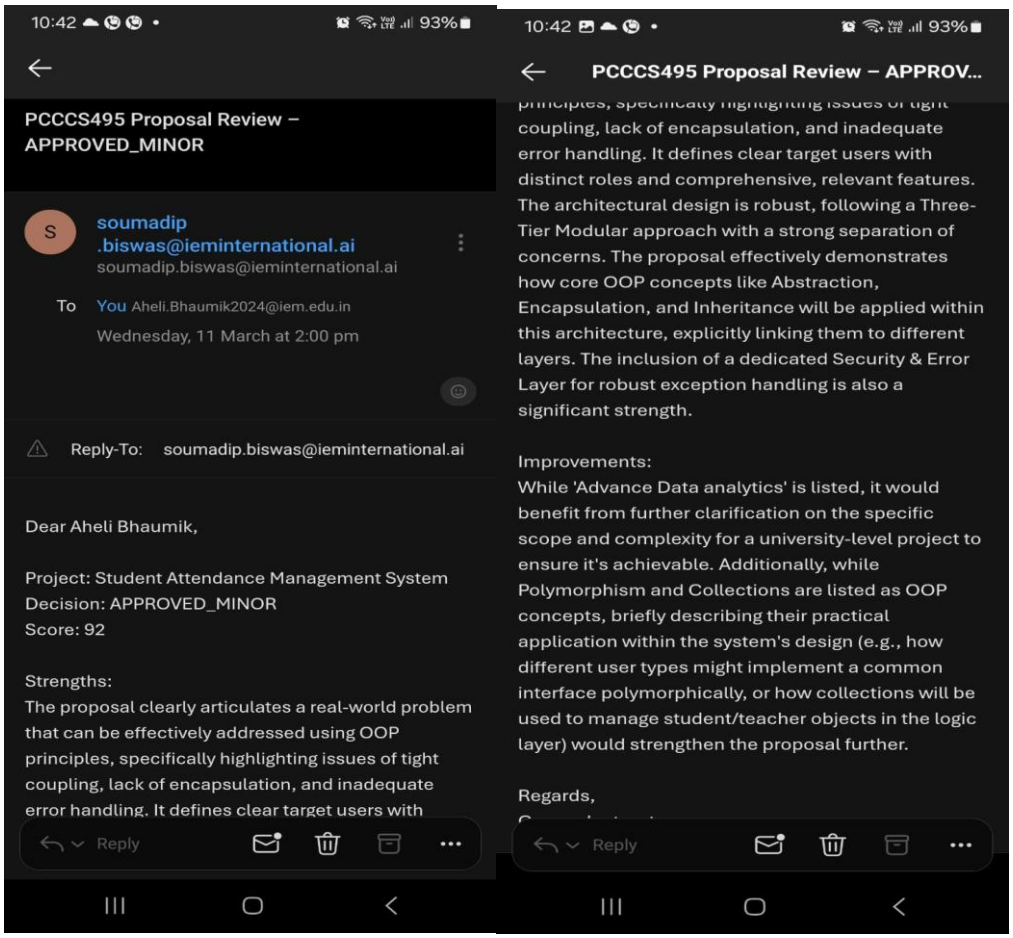
Advanced Data Analytics: Logic-based reporting, where attendance percentage is determined for each class, along with the identification of "At-Risk" students based on the attendance percentage, such as students with less than a 75% attendance rate.

Robust Input Validation: Extensive use of Error Handling using try-catch blocks to prevent the system from crashing due to invalid IDs, duplicate records, or invalid data files.

Persistent File Management: A separate Data Access Layer for storing and retrieving data, ensuring the data is saved across system shutdowns, without the use of the "CRUD-only" hardcoded method.

Search & Filter Engine: A high-speed search function for students based on Roll Number, Name, or Semester, using optimized data structures.

COPY OF REVIEW MAIL



PCCCS495 Proposal Review – APPROVED_MINOR

soumadip.biswas@ieminternational.ai

soumadip.biswas@ieminternational.ai

To You Aheli.Bhaumik2024@iem.edu.in

Wednesday, 11 March at 2:00 pm

Reply-To: soumadip.biswas@ieminternational.ai

Dear Aheli Bhaumik,

Project: Student Attendance Management System
Decision: APPROVED_MINOR
Score: 92

Strengths:
The proposal clearly articulates a real-world problem that can be effectively addressed using OOP principles, specifically highlighting issues of tight coupling, lack of encapsulation, and inadequate error handling. It defines clear target users with distinct roles and comprehensive, relevant features. The architectural design is robust, following a Three-Tier Modular approach with a strong separation of concerns. The proposal effectively demonstrates how core OOP concepts like Abstraction, Encapsulation, and Inheritance will be applied within this architecture, explicitly linking them to different layers. The inclusion of a dedicated Security & Error Layer for robust exception handling is also a significant strength.

Improvements:
While 'Advance Data analytics' is listed, it would benefit from further clarification on the specific scope and complexity for a university-level project to ensure it's achievable. Additionally, while Polymorphism and Collections are listed as OOP concepts, briefly describing their practical application within the system's design (e.g., how different user types might implement a common interface polymorphically, or how collections will be used to manage student/teacher objects in the logic layer) would strengthen the proposal further.

Regards,